SREERAG E G

Full Stack Software Engineer

Java | Python | ReactJS | AWS | Event-Driven Microservices | AI-Assisted Development

Thrissur, Kerala, India | +91 9567870963
sreerageg@gmail.com | linkedin.com/in/sreerag-e-g | github.com/sreerageg

SUMMARY

Full stack software engineer who owns systems end to end — backend services and REST APIs, front-end integration, and event-driven distributed workloads on AWS. Primary author (roughly 73% of commits) and technical owner of the production platform running US healthcare copay-assistance, claim submission and revenue-cycle workflows: 233 modules across 40+ payer and manufacturer portals, orchestrated over queue-driven Celery/SQS pipelines into containerized AWS ECS worker pools, under HIPAA and PHI controls. Drives AI-assisted development across the SDLC — built the in-house LLM agent pipeline that generates, reviews and standardizes production code, with every AI-generated output gated by peer review, automated testing and secure-coding standards. Sets framework conventions other engineers build against, leads design-level discussions, and works directly with business stakeholders in a regulated, compliance-heavy domain.

TECHNICAL SKILLS

- **Languages:** Java (OOP, collections, generics, concurrency), Python (advanced OOP, framework & API design), JavaScript (ES6+), SQL, HTML/CSS.
- **Backend & Frameworks:** Spring Boot (REST controllers, dependency injection, Spring Data JPA, Spring Security), FastAPI, Django REST Framework; microservices design, service decomposition, API versioning, OAuth 2.0 / JWT, Swagger/OpenAPI.
- **Frontend:** ReactJS (functional components, hooks, state management, REST/JSON integration), responsive UI, cross-browser DOM behaviour.
- **Cloud (AWS):** ECS (containerized worker pools), SQS, S3, Lambda, EC2, IAM roles & least-privilege policies, CloudWatch logging; cost- and workload-aware service sizing.
- **DevOps & Tooling:** Docker, Kubernetes (working knowledge), GitHub Actions CI/CD, Git branching & code review workflows, Maven/Gradle, JUnit & Mockito, pytest, Jira/Agile boards.
- **Data & Messaging:** MySQL, PostgreSQL, MongoDB; schema design, query optimization, large-dataset analysis; event-driven architecture with Celery/SQS (Kafka concepts), producer/consumer patterns, idempotent message handling, dead-letter and retry semantics.
- **AI Engineering:** LLM agent pipelines for code generation, refactoring and automated review; GitHub Copilot / Claude Code / Cursor in daily workflow; spec & prompt design, structured extraction, evaluation of AI output; responsible AI use — data-sensitivity screening, PHI-safe inputs, mandatory human validation.
- **Reliability & Security:** Typed error classification, retry & exponential backoff, idempotency, graceful degradation, application resiliency patterns, PHI masking, HIPAA compliance, secure coding standards.
- **Automation & Extraction:** Playwright, Selenium, Page Object Model, self-healing selectors, PDF/HTML parsing and structured data extraction.
- **Practices:** Agile/Scrum, full SDLC ownership, design reviews, peer code review, unit & integration testing, CI/CD delivery, technical documentation, stakeholder communication.

PROFESSIONAL EXPERIENCE

Automation Platform Engineer | GetCopayHelp (Kalki Safalya Consulting Services LLP) *May 2025 – Present*

- **AI-Assisted Development Pipeline:** Built roughly 2,100 lines of LLM agent tooling that converts recorded browser sessions into framework-compliant production code — selecting the base class, placing the module, wiring queue registration and API routing, and enforcing review rules — collapsing new-portal onboarding from days to hours.
- **Responsible AI in the SDLC:** Use AI coding assistants daily for code generation, refactoring, unit-test creation and documentation, with every output validated through peer review, automated tests and secure-coding checks before merge; enforce data-sensitivity rules so no PHI or credential material ever enters a prompt.
- **Event-Driven Microservices on AWS:** FastAPI service gateway routes 100+ daily runs across 4 Celery/SQS queues into containerized AWS ECS worker pools sized by workload class, sustaining 500+ claim submissions weekly with horizontal scaling, idempotent retries and isolation between workload types.
- **Platform & Framework Architecture:** Designed the shared OOP framework behind all 233 automation modules, modelling 8 distinct workflow types (enrollment, claim submission, claim retrieval, payment fetch, consent, waitlist, verification) as first-class base classes and replacing per-script duplication with a single extensible core.

- **API Design & System Integration:** Own the REST contracts between the orchestration gateway, worker services and downstream billing/reporting consumers, designing each change with explicit consideration of upstream payer portals and downstream revenue-cycle systems.
- **Application Resiliency:** Built a typed error registry making retryability, severity and remediation properties of a classified error type rather than ad-hoc call-site handling — cutting maintenance overhead by 30% and making cross-system failures observable and triageable.
- **Third-Party Integration & 2FA:** Designed a portal-agnostic email-OTP retrieval layer with watermark-based freshness and consume-once semantics, restoring automated login after payer portals migrated to Okta two-factor authentication.
- **Secure SDLC & Compliance:** Enforced PHI masking at the logging boundary through typed maskers and a logging filter, so no service can leak patient data through logs regardless of call site; ship through GitHub Actions CI/CD with automated checks and review gates.
- **Data-Driven Prioritization:** Analyze run-outcome and error-class data across 100+ daily executions and 40+ portals to surface failure patterns, quantify impact and prioritize engineering work against business throughput.
- **Technical Leadership:** Define the framework conventions every contributor builds against, review incoming changes for correctness and security, and onboard new engineers onto the platform architecture.

Full Stack Developer (Intern) | Aspire Systems *Nov 2024 – May 2025*

- Built and optimized 12+ REST endpoints using Django REST Framework and MySQL, improving query response times by 20% for enterprise applications.
- Integrated backend services with ReactJS front ends, working through JSON contracts, error states and authenticated requests alongside the UI team.
- Implemented OAuth 2.0 authentication flows (including Google OAuth) onboarding 200+ internal users; automated API documentation with Swagger.
- Delivered in two-week Agile sprints within a large corporate engineering environment — story estimation, peer code review, and defect triage across the full SDLC.

Freelance Python Developer (Automation) | Self-Employed *May 2024 – Nov 2024*

- **Legacy Framework Migration:** Restructured 30+ legacy Selenium scripts into a modular Page Object Model framework, reducing script failure rates by 50%.
- Worked directly with clients to gather requirements, scope deliverables and present technical trade-offs, then delivered and supported each engagement end to end.

EDUCATION

- **B.Tech, Computer Science and Engineering** | Christ College of Engineering 7.85 CGPA